

# TTT-VLA：视觉-语言-动作模型的测试时潜在提示优化

张文波<sup>1,2,\*</sup>、李建雄<sup>1,3</sup>、杨帅<sup>4</sup>、陈思进<sup>1,5</sup>、刘家军<sup>6</sup>、刘灵巧<sup>2,†</sup>、马小<sup>1,†</sup>

<sup>1</sup>ByteDance Seed、<sup>2</sup>阿德莱德大学、<sup>3</sup>清华大学、<sup>4</sup>浙江大学、<sup>5</sup>香港大学、<sup>6</sup>CSIRO Data61

\*在字节跳动种子完成的工作，†通讯作者

## 抽象的

在大规模数据上训练的视觉-语言-动作（VLA）模型已经取得了显著的进步，但它们仍然容易受到部署时分布变化的影响。最近的 VLA 模型表明，提示可以作为指导政策行为的有效界面，但现有的基于提示的指导通常依赖于外部指导。这就提出了一个自然的问题：能否通过优化提示来实现 VLA 的测试时训练（TTT），从而使转向界面本身可以从交互中学习和适应？我们使用 TTT-VLA 来解决这个问题，这是一种基于潜在提示优化（LPO）的测试时训练框架。在训练过程中，通过额外的代理任务学习潜在提示，为策略学习提供额外的学习调节信号。在测试时，TTT 是通过从当前环境收集交互数据并使用代理任务的自监督信号仅优化这些数据的潜在提示来执行的，而不修改策略本身。SimplerEnv 上的实验表明，所提出的方法在单实施例和多实施例设置中都能持续提高任务成功率。进一步的分析表明，收益主要来自纠正少数关键决策，而不是在全球范围内改变政策行为。这些结果表明，LPO 为基础操纵策略的部署时间改进提供了有效且实用的途径。

日期：2026 年 6 月 1 日

科雷斯波 来源: xiao.ma@bytedance.com, lingqiao.liu@adelaide.e

杜.au

## 1 简介

视觉-语言-动作（VLA）模型是通用机器人控制的一个有前途的框架，因为它们将视觉感知、语言调节和动作生成统一在一个策略中。对 VLA 更好泛化的追求推动了一系列技术方向的发展，包括大规模预训练 [1-3, 12, 21]、潜在动作模型 [5, 38] 和基于世界模型的方法 [13, 39]。实现这一目标的另一个实用途径是改进部署时的策略，特别是当模型必须在视觉条件、环境或实施例的变化下运行时。然而，改善部署时间的主要方法仍然是强化学习（RL），它可以通过交互来改进策略，但通常成本高昂且依赖于奖励[22, 25]。

与此同时，最近的研究表明，基于提示的转向正在成为一个重要的界面

VLA 中部署时间的改进 [14,26,40,41]。特别是，借助因子化指令等人工协助，现成的策略可以实现更好的性能。虽然有用，但此范例无法从部署经验中学习。另一方面，正如  $\pi_{0.7}$  所强调的，任务的成功和熟练程度也可能取决于难以仅用文本表达的更细粒度的上下文，例如情节质量、执行速度或视觉提示 [26]。这些因素反映了与控制相关的时间和空间背景，表明 VLA 控制的此类背景的表达方式仍有很大的改进空间。

受此观察的启发，我们探索了部署时改进的新方向：在训练期间学习潜在提示  $z$  以调节策略，然后在测试时仅通过调整  $z$  来改进策略。我们使用 TTT-VLA 来解决这个问题，这是一个基于潜在提示优化的框架。如图1所示，核心思想是从与测试环境的交互中获得改进信号，并用它来优化潜在提示。由于 VLA 经过训练以适应此提示，因此可以通过轻量级接口吸收测试时知识，而无需修改策略主干。在我们当前的实现中，改进信号来自称为状态接地的代理任务，它使用观察状态监督来塑造潜在提示。这个代理任务鼓励潜在提示捕获空间相关信息[35]，我们进一步设计训练策略来加强这种效果。由此产生的拉-

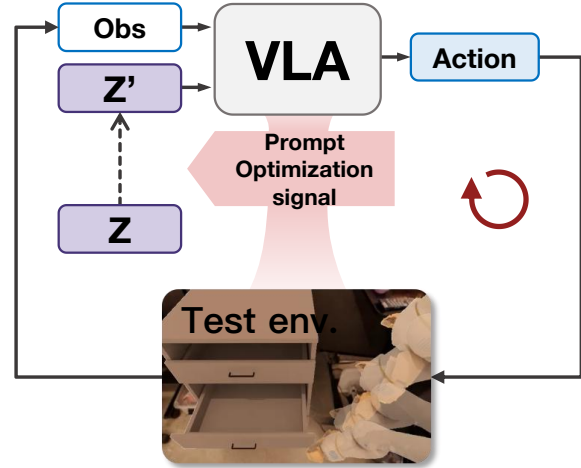


图 1 具有潜在提示优化的 TTT-VLA 概述。

帐篷提示有两个作用：在训练期间，它为政策学习提供额外的习得条件信号；在测试时，收集当前环境中的交互数据并用于仅更新冻结策略的潜在提示。

我们在 SimplerEnv [15] 中在单实施例和多实施例设置下评估了这个想法。我们的评估遵循官方协议，其中策略在真实机器人数据上进行训练并在模拟中进行测试，自然引入了部署时域间隙。在单实施例设置中，我们对 WidowX 和 Google Robot 任务进行评估。在多实施例设置中，我们在 OXE-Aug [10] 的 BridgeData V2 [30] 上训练九个实施例，然后在 WidowX 上进行评估。在这些设置中，我们通过 TTT-VLA 实现了一致的改进，同时保持主要策略不变。分析进一步表明，大部分收益来自我们所谓的关键决策指导，其中测试时训练

表 1 VLA 部署时间改进策略的比较。这些策略的不同之处在于它们在部署时如何干预策略、更新什么以及推动改进的信号。TTT-VLA 属于自监督测试时间训练体系：它仅优化潜在提示，同时保持策略主干冻结。更多比较参见5。

|                               | Human-Assisted Steering                                     | RL Post-Training        | Test-Time Training                                                            |
|-------------------------------|-------------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------|
| <b>Intervention interface</b> | Prompts, e.g., task-factorized instruction, metadata, trace | Typically policy itself | Typically parts of the parameters. In this paper, we only use latent prompts. |
| <b>Improvement signal</b>     | Human guidance / external signal                            | Reward                  | Self-supervised loss                                                          |
| <b>Policy backbone status</b> | Frozen                                                      | Generally updated       | Frozen                                                                        |

纠正少量关键动作，而不是全局重写完整轨迹。我们的贡献总结如下：

- TTT-VLA是基于潜在提示优化的VLA部署时改进框架，它在训练期间学习潜在提示，并在部署时对冻结策略执行仅提示测试时训练。
- 单实施例和多实施例设置中一致的部署时间改进，表明潜在提示可以作为从交互中学习的有效界面。
- 对潜在提示的有效培训和更新策略的调查表明，收益主要来自我们称之为关键决策指导的现象。

## 2 相关工作

### 2.1 操纵策略的部署时间改进

通过强化学习后训练、人工辅助提示转向以及最近的测试时训练，研究了操纵策略的部署时间改进。一项既定的工作通过从部署经验中进行强化学习来改进 VLA 策略。代表性的例子包括  $\pi_{0.6}^*$ ，它研究基于异构经验和人类修正的基于 RL 的自我改进，以及 GR-RL，它通过多阶段强化学习将通才 VLA 变成灵巧的专家 [22, 25]。

另一条工作线使用 VLA 调节接口本身作为部署时人工协助的渠道。Yell At Your Robot 在执行过程中使用实时语言校正，而 MolmoAct 和 TraceVLA 将可编辑轨迹公开为结构化转向信号，用于时空指导 [14,28,41]。 $\pi_{0.7}$  通过元数据、语言指导和图像目标上下文进一步拓宽了这个接口，表明人类或外部指定的上下文可以提高轨迹质量和对未见过的任务的泛化 [26]。

操纵政策测试时训练的早期工作包括 PAD [7]，它在小规模政策上表明，自我监督信号可以帮助操纵政策适应环境变化。然而，PAD 并未在基础模型体系中进行研究。因此，它的设计和结论都没有直接转移到现代 VLA 模型。最近，测试时训练开始出现在 VLA 文献中。同期的一项工作是 WorldAgen [31]，它联合学习动作预测和世界建模，然后在部署时根据新收集的转换来调整其世界模型分支。然而，其测试时训练方案是围绕与该方法一起引入的定制架构构建的，而不是围绕强大的最先进的 VLA 模型构建的。相比之下，我们研究提示作为预训练 VLA 模型的轻量级适应接口，以便测试时训练是在具有竞争力的性能的强大基础策略之上执行的。

### 2.2 即时条件VLA

最近的 VLA 模型越来越多地围绕提示条件界面进行组织，这些界面超出了语言本身的范围。RT-2、OpenVLA 和 Octo 等早期系统建立了语言条件机器人控制，Octo 进一步表明子目标图像可以作为额外的任务规范通道 [3,11,20,37]。最近的工作通过使用结构化轨迹或跟踪输入作为提示进一步拓宽了提示，如 MolmoAct 和 TraceVLA [14, 41]。 $\pi$  系列进一步扩展了这条线： $\pi_0$  提高了语言条件控制的广度和灵活性， $\pi_{0.5}$  通过分层子任务指令增强了提示，而  $\pi_{0.7}$  通过从世界模型派生的元数据和图像目标上下文进一步拓宽了条件空间 [1,24,26]。这些作品共同表明，更丰富的提示界面是现代 VLA 性能的核心。我们的方法是对这条线的补充：我们不是添加另一个人类指定的提示通道，而是学习一个潜在提示，该提示捕获 VLA 策略的控制相关上下文，并且可以通过测试时训练进行更新。

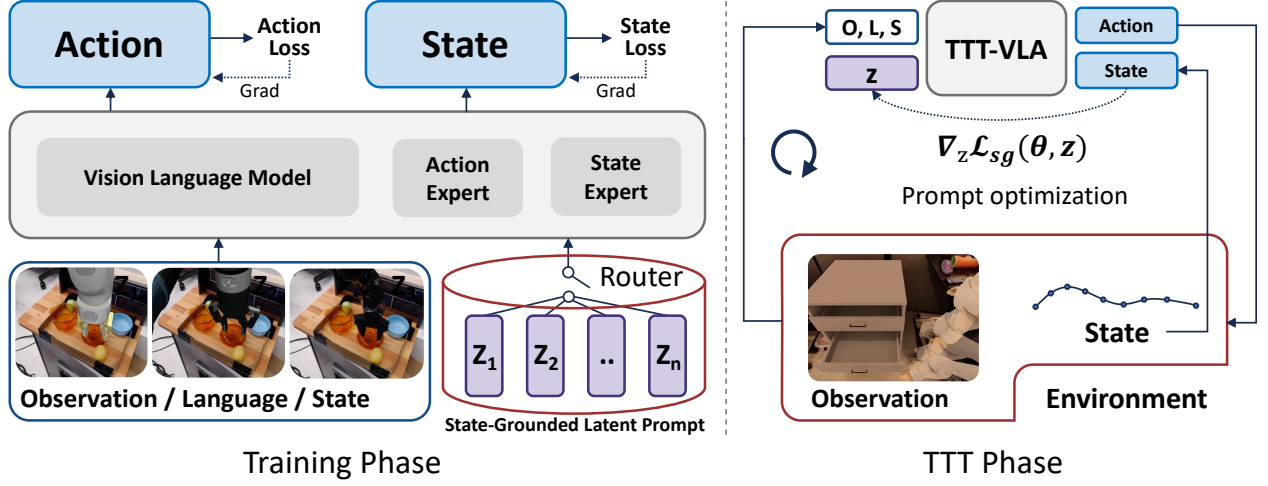


图 2 TTT-VLA 概述。该策略以当前观察、显式提示和潜在提示为条件。在训练过程中，潜在提示与策略和状态接地代理任务一起进行优化。在部署期间，测试时训练仅更新与环境交互的潜在提示，而不修改策略本身。

### 方法3：潜提示优化

一般来说，VLA 策略可以写为条件策略，它将当前观察  $o$  与条件上下文  $c$  结合起来，并通过以下方式预测动作块  $a$

$$a \sim \pi_{\theta}(a \mid o, c), \quad (1)$$

其中  $c$  是统一条件变量。它可以包括具有可读语义的显式条件，例如语言、状态、图像或其他结构化上下文，以及可学习的潜在提示  $z$  形式的隐式条件。在我们当前的实现中， $z \in \mathbb{R}^{n \times d}$  被实例化为  $n$  提示标记的可学习块，其中  $d$  是策略主干的隐藏维度， $n$  是控制提示长度的超参数。遵循基于流的 VLA 公式，我们将动作生成建模为动作块上的条件流匹配。令  $a_0 \sim p_0$  表示动作空间中的噪声样本，令  $a_1 = a$  表示目标动作块。对于插值时间  $t \sim \mathcal{U}(0, 1)$ ，我们将噪声动作轨迹定义为  $a_t = (1 - t)a_1 + ta_0$ ，相应的目标向量场为  $u_t = a_0 - a_1$ 。该策略预测以当前输入和完整调节上下文为条件的向量场：

$$\mathcal{L}_{\text{act}}(\theta, z) = \mathbb{E}_{(o, c, a), a_0, t} \left[ \|v_{\theta}(a_t, t; o, c) - u_t\|_2^2 \right]. \quad (2)$$

该目标定义了我们框架中的基本政策学习问题。潜在提示优化不会取代行动目标。相反，它添加了一个学习调节变量，可以在训练期间进行优化，然后在部署时进行调整。下面的小节介绍了如何在部署时通过抽象代理目标更新此潜在提示，后面的实现小节指定了我们当前系统中使用的具体代理。

#### 3.1 通过潜在提示优化进行测试时训练

受到先前测试时训练和部署时自监督适应工作 [6,7,29] 的启发，我们引入了代理任务来启用测试时训练以实现潜在提示优化。由于行动目标不能直接用于部署时的仅提示适应，因此该代理任务提供了执行测试时训练过程所需的辅助学习目标。同时，它在训练期间将潜在提示与控制相关上下文结合起来，以便稍后可以重复使用相同的目标来进行部署时间调整。在训练过程中，我们使用行动目标和代理目标来优化策略，以便潜在提示对于行动预测有用，同时通过以下方式保持适应性：

the same proxy signal:

$$\mathcal{L}_{\text{train}}(\theta, z) = \mathcal{L}_{\text{act}}(\theta, z) + \mathcal{L}_{\text{proxy}}(\theta, z). \quad (3)$$

在测试时，我们的主要程序是从当前环境交互缓冲区进行仅提示的适应。我们首先从当前环境中收集交互数据，将生成的样本组织到缓冲区中，然后仅更新潜在提示，同时保持策略主干冻结。由此产生的测试时训练规则可以写为

$$z \leftarrow z - \eta \nabla_z \mathcal{L}_{\text{proxy}}, \quad (4)$$

使用当前环境中的交互数据。这里， $\mathcal{L}_{\text{proxy}}$ 表示训练中引入的相同代理目标，其具体实例稍后作为第2节中的状态基础任务给出。3.2.这保留了明确的分工：动作专家仍然负责动作生成，而潜在提示是部署期间唯一适应的变量。我们还探索在线提示更新作为消融，但在我们当前的实验中，这种模式不稳定并且提示表示往往会崩溃，因此主要方法使用上面的离线缓冲区公式。

### 3.2 架构和训练策略

网络结构。如图2所示，我们当前的实现使用变压器混合（MoT）架构[1]实例化TTT-VLA，该架构由两个流匹配专家组成：一个预测动作空间中的流场的动作专家，和一个预测状态空间中机器人状态接地的流目标的状态接地专家。我们考虑使用硬路由器来进行潜在提示选择。如果训练数据包含多个实施例，则随机初始化多个潜在提示。在单实施例设置中，这种设计自然退化为单个潜在提示。

详细的注意力掩模如图3所示。动作专家是否能够注意到潜在提示取决于具体的训练策略，稍后将对此进行描述。在训练期间，参数从预训练的 $\pi_{0.5}$ 检查点初始化。请注意，状态输入略有不同：在执行 $\pi_0$ 后，状态输入从语言形式条件转移到动作专家。引入此更改主要是为了避免与我们的代理任务发生冲突。我们的主要比较集中在两个变体上。

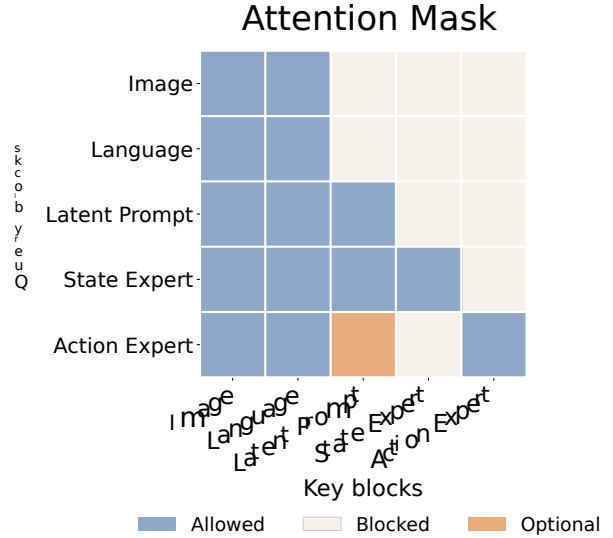


图3 MoT网络中使用的注意力掩模。

状态接地作为代理任务。在我们当前的实现中，潜在提示优化是通过称为状态接地的代理任务来实例化的，其中专门的专家根据当前观察、显式提示和潜在提示来预测机器人末端执行器位置和夹具状态。令 $s_1 = s$ 表示目标机器人状态，由末端执行器位置和夹具状态组成，令 $s_0 \sim p_0^s$ 表示状态空间中的噪声样本。

对于插值时间 $t \sim \mathcal{U}(0, 1)$ ，我们将噪声状态轨迹定义为 $s_t = (1 - t)s_1 + ts_0$ ，相应的目标向量场为 $u_t^s = s_0 - s_1$ 。状态接地专家根据当前输入和完整的条件上下文来预测该向量场：

$$\mathcal{L}_{\text{proxy}}(\theta, z) = \mathcal{L}_{\text{sg}}(\theta, z) = \mathbb{E}_{(o, c, s), s_0, t} \left[ \|v_\theta^s(s_t, t; o, c) - u_t^s\|_2^2 \right]. \quad (5)$$

我们使用此任务作为代理目标有两个原因。首先，这些状态目标在训练和部署时都可用，无需额外的注释成本，使它们成为实用的自监督信号。第二，

准确的状态预测需要对场景的空间理解以及对机器人特定实施例的理解，因此提供丰富的控制相关线索[31, 35]。剩下的问题是如何确保这些线索比其他参数更有效地吸收到潜在提示中，从而激发下面的训练策略。

单实施例和多实施例设置的训练策略。在多实施例设置中，实施例提供了异构机器人数据到不同的控制相关上下文的自然划分。因此，我们将不同的基于状态的潜在提示与不同的实施例相关联，并与行动目标和基于状态的目标共同优化它们。这种设计鼓励跨实施例的控制差异被吸收到潜在提示中，而不是完全纠缠在共享模型参数中。

单实施例设置更受数据限制，因此这种自然划分变得更弱，并且必须更明确地强制执行即时学习。为了解决这个问题，我们使用两阶段训练策略。如图3所示，从潜在提示到动作专家的连接是可选的：在第一阶段，它以概率1被阻止，在第二阶段，它以概率0.5被阻止。附录图 8 提供了这种随机下降设计的消除。如果没有随机下降，简单地添加潜在提示就可能降低基本策略性能，这表明更强的脱钩在单一实施方案中很重要。相关的提示下降或条件下降策略也已在先前的工作中使用过[9]。此外，我们将状态基础目标的梯度流限制为状态专家和潜在提示，而动作目标的梯度流则限制为VLM主干和动作专家。总之，这些约束鼓励潜在提示在与动作预测更紧密地耦合之前从代理任务中吸收与控制相关的上下文。

表 2 在 SimplerEnv WidowX 基准上与最先进策略的性能比较。对于最后三行，所有行都是从预训练的  $\pi_{0.5}$  检查点初始化的。这里，“ $\pi_{0.5}$ ”表示我们的内部基线，“ $\pi_{0.5} + \text{SG-LP}$ ”添加了带有潜在提示训练的状态基础，“ $\pi_{0.5} + \text{SG-LP} + \text{TT}$ ”进一步将测试时训练应用于学习的潜在提示。

| WidowX Benchmark                        | Carrot       | Eggplant     | Spoon        | Cube         | Overall      |
|-----------------------------------------|--------------|--------------|--------------|--------------|--------------|
| RT-1-X [4]                              | 4.2%         | 0%           | 0%           | 0%           | 1.1%         |
| OpenVLA [12]                            | 0%           | 4.1%         | 0%           | 0%           | 1.0%         |
| SpatialVLA [27]                         | 20.8%        | 70.8%        | 20.8%        | 25.0%        | 34.4%        |
| Magma [36]                              | 29.2%        | 91.7%        | 37.5%        | 20.8%        | 44.8%        |
| Octo-Base [20]                          | 8.3%         | 43.1%        | 12.5%        | 0%           | 16.0%        |
| Octo-Small [20]                         | 9.7%         | 56.9%        | 47.2%        | 4.2%         | 29.5%        |
| RoboVLM [18]                            | 20.8%        | 79.2%        | 45.8%        | 4.2%         | 37.5%        |
| InstructVLA [37]                        | 40.3%        | <b>94.4%</b> | 43.1%        | 9.7%         | 46.9%        |
| $\pi_0$ [1]                             | 36.1%        | 81.9%        | 45.8%        | 26.4%        | 47.6%        |
| CogACT [16]                             | 37.5%        | 91.7%        | 58.3%        | 20.8%        | 52.1%        |
| ThinkAct [8]                            | 37.5%        | 70.8%        | 58.3%        | 8.7%         | 43.8%        |
| $\pi_{0.5}$                             | <b>79.2%</b> | 75.0%        | 33.3%        | 16.7%        | 51.1%        |
| $\pi_{0.5} + \text{SG-LP}$              | 69.5%        | 70.5%        | <b>72.5%</b> | 41.5%        | 63.5%        |
| $\pi_{0.5} + \text{SG-LP} + \text{TTT}$ | 74.5%        | 76.0%        | 71.0%        | <b>48.0%</b> | <b>67.4%</b> |

## 4 实验

基准。我们使用 SimplerEnv[15] 作为本节中的评估基准。在此基准测试中，我们研究了单实施例和多实施例设置：前者涵盖固定实施例 WidowX 和 Google Robot 任务，而后者研究 OXE-Aug（增强的开放 X-实施例（OXE）训练集）的训练以及 WidowX 的评估。为了便于阅读，表 2、4、7 和图 6 中的 WidowX 任务缩写为 Spoon、Carrot、Cube 和 Eggplant，分别对应于将勺子放在毛巾上、将胡萝卜放在盘子上、堆叠块和将茄子放入篮子中。

基线。为了进行比较，我们报告了 SimplerEnv 上一系列现有方法的公开最新结果。

表 3 SimplerEnv Google Robot 任务的性能比较。报告视觉匹配和变体聚合协议的结果。

| Visual Matching                 |               |              |                   |              |
|---------------------------------|---------------|--------------|-------------------|--------------|
| Model                           | Pick Coke Can | Move Near    | Open/Close Drawer | Avg          |
| HPT [33]                        | 56.0%         | 60.0%        | 24.0%             | 46.0%        |
| TraceVLA [41]                   | 28.0%         | 53.7%        | 57.0%             | 42.0%        |
| RT-1-X [2]                      | 56.7%         | 31.7%        | 59.7%             | 53.4%        |
| Octo-Base [20]                  | 17.0%         | 4.2%         | 22.7%             | 16.8%        |
| OpenVLA [12]                    | 16.3%         | 46.2%        | 35.6%             | 27.7%        |
| RoboVLM [18] (zero-shot)        | 72.7%         | 66.3%        | 26.8%             | 56.3%        |
| RoboVLM [18] (fine-tuned)       | 77.3%         | 61.7%        | 43.5%             | 63.4%        |
| Emma-X [17]                     | 2.3%          | 3.3%         | 18.3%             | 8.0%         |
| $\pi_0$ [1] (fine-tuned)        | 72.7%         | 65.3%        | 38.3%             | 58.7%        |
| $\pi_0$ -Fast [23] (fine-tuned) | 75.3%         | 67.5%        | 42.9%             | 61.9%        |
| GR00T [19] (fine-tuned)         | 69.3%         | 68.7%        | 35.8%             | 52.4%        |
| $\pi_{0.5}$                     | 84.0%         | 59.2%        | 59.3%             | 67.5%        |
| $\pi_{0.5}$ + SG-LP             | <b>85.0%</b>  | 66.2%        | 55.6%             | 68.9%        |
| $\pi_{0.5}$ + SG-LP + TTT       | <b>85.0%</b>  | <b>71.7%</b> | <b>60.6%</b>      | <b>72.4%</b> |

| Variant Aggregation             |               |              |                   |              |
|---------------------------------|---------------|--------------|-------------------|--------------|
| Model                           | Pick Coke Can | Move Near    | Open/Close Drawer | Avg          |
| TraceVLA [41]                   | 60.0%         | 56.4%        | 31.0%             | 45.0%        |
| RT-1-X [2]                      | 49.0%         | 32.3%        | 29.4%             | 39.6%        |
| Octo-Base [20]                  | 0.6%          | 3.1%         | 1.1%              | 1.1%         |
| OpenVLA [12]                    | 54.5%         | 47.7%        | 17.7%             | 39.8%        |
| RoboVLM [18] (zero-shot)        | 68.3%         | 56.0%        | 8.5%              | 46.3%        |
| RoboVLM [18] (fine-tuned)       | 75.6%         | 60.0%        | 10.6%             | 51.3%        |
| Emma-X [17]                     | 5.3%          | 7.3%         | 20.5%             | 11.0%        |
| $\pi_0$ [1] (fine-tuned)        | 75.2%         | 63.7%        | 25.6%             | 54.8%        |
| $\pi_0$ -Fast [23] (fine-tuned) | 77.6%         | <b>68.2%</b> | 31.3%             | 59.0%        |
| GR00T [19] (fine-tuned)         | 46.7%         | 62.9%        | 17.5%             | 43.7%        |
| $\pi_{0.5}$                     | <b>82.0%</b>  | 47.7%        | 44.7%             | 58.1%        |
| $\pi_{0.5}$ + SG-LP             | 81.7%         | 51.2%        | 42.9%             | 58.6%        |
| $\pi_{0.5}$ + SG-LP + TTT       | 79.3%         | 55.2%        | <b>45.8%</b>      | <b>60.1%</b> |

作为对我们方法的严格控制，我们包括一个没有状态专家或潜在提示的基线，这是直接从原始  $\pi_{0.5}$  微调的版本。第二个符号是“ $\pi_{0.5}$  + SG-LP”（状态接地潜在提示），它通过 SG-LP 训练扩展基线。第三个是“ $\pi_{0.5}$  + SG-LP + 测试时训练（TTT）”，它将测试时训练进一步应用于学习到的潜在提示。后一个变体是我们的完整方法，TTT-VLA。

训练。所有模型都是从预训练的  $\pi_{0.5}$  检查点初始化的。在实现  $\pi_0$  之后，我们将状态输入从语言形式条件转移到动作专家中，以避免与代理任务发生冲突。在单实施例设置中，模型经过 20,000 个步骤的训练；在多实施例设置中，它们接受了 40,000 个步骤的训练。我们使用 AdamW，学习率为  $1 \times 10^{-4}$ ，批量大小为 1024。所有训练运行均在 32 个 NVIDIA H100 GPU 上进行，每次运行的挂钟时间约为 20 小时。

测试时培训。在测试时，我们仅更新潜在提示，从训练期间学到的预训练提示对其进行初始化。我们使用的学习率为  $1 \times 10^{-5}$ ，批量大小为 128。除非另有说明，我们对 WidowX 运行 500 个优化步骤，对 Google Robot 任务运行 1000 个优化步骤。我们不会根据评估表现来选择最佳检查点；相反，我们提前为每个基准确定优化范围，以便该过程在每个设置内保持一致，并在之后停止



表 4 SimplerEnv WidowX 任务的多实施例评估。该策略在 OXE-Aug Bridge V2 上进行训练，总共有九个实施例，并在四个 Bridge V2 WidowX 任务上进行评估。

| Method                                  | Carrot       | Eggplant     | Spoon        | Cube        | Mean         |
|-----------------------------------------|--------------|--------------|--------------|-------------|--------------|
| $\pi_{0.5}$                             | 28.0%        | 17.5%        | 43.0%        | <b>2.5%</b> | 22.8%        |
| $\pi_{0.5} + \text{SG-LP}$              | 41.0%        | 29.5%        | 43.0%        | 0.5%        | 28.5%        |
| $\pi_{0.5} + \text{SG-LP} + \text{TTT}$ | <b>43.0%</b> | <b>34.0%</b> | <b>48.5%</b> | 1.0%        | <b>31.6%</b> |

代理损失已基本趋于稳定。测试时训练在 8 个 NVIDIA H100 GPU 上进行，通常需要 15-30 分钟的挂钟时间。附录 6 提供了更多详细信息。

评估协议。SimplerEnv 中的评估是随机的，因为 (1) 动作是通过流匹配采样生成的，(2) WidowX 基准测试在具有不同对象位置和方向的四个任务中只有 24 种配置。因此，单次扫描的噪声太大，无法进行可靠的比较。因此，按照 GR00T [19]，我们通过循环 24 种配置来评估 200 集的每个 WidowX 任务，同时允许采样噪声在试验中产生不同的动作。在模型变体中，我们仅修复初始种子而不是完整的下游随机流，因此采样的噪声和结果动作仍然不同。这减少了评估方差并支持更可靠的比较。对于 Google Robot 视觉匹配和变体聚合，我们使用官方默认评估计数。优化评估流程后，Bridge 的一次完整评估的挂钟时间约为 40 分钟，分形视觉匹配约为 50 分钟，分形变体聚合约为 3 小时

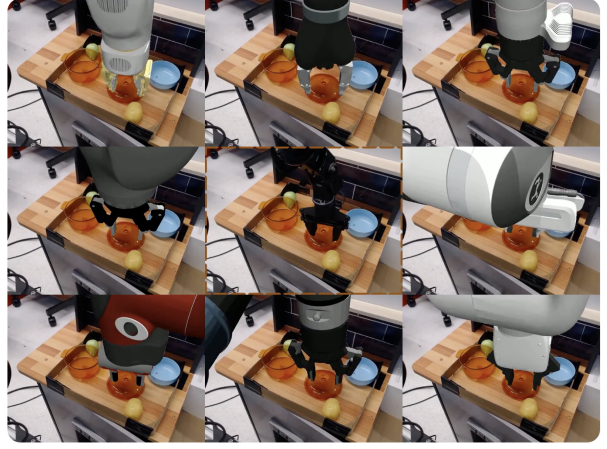


图 4 跨九个训练实施例的 OXE-Aug Bridge V2 示例。

#### 4.1 单实施例设置

我们首先研究单实施例设置，其中主要问题是即使实施例身份本身固定，学习到的潜在提示是否有用。表 2 显示了 SimplerEnv WidowX 基准测试的结果。向  $\pi_{0.5}$  添加潜在提示优化将平均成功率从 51.1% 提高到 63.5%，测试时训练将其进一步提高到 67.4%。我们在表 3 中观察到 Google Robot 任务的相同趋势。在视觉匹配分割中，平均成功率从  $\pi_{0.5}$  的 67.5% 提高到使用潜在提示优化的 68.9%，然后在测试时训练后提高到 72.4%。在变体聚合拆分中，相同的序列从 58.1% 提高到 58.6%，然后提高到 60.1%。

#### 4.2 多实施例设置

我们接下来研究多实施例设置。这里，训练数据来自 OXE-Aug Bridge V2，总共包含 9 个实施例：原始 WidowX 加上 8 个附加实施例，即 Panda、UR5e、Xarm7、Google Robot、Sawyer、Kinova3、IIWA 和 Jaco，如图 4 所示。在评估过程中，路由器在推理之前为 WidowX 选择相应的提示。结果如表 4 所示。与  $\pi_{0.5}$  基线相比，潜在提示优化将平均成功率从 22.8% 提高到 28.5%，测试时训练将其进一步提高到 31.6%。改进主要集中在胡萝卜和茄子上，而立立方体对于所有变体来说仍然很困难。根据经验，我们发现 Cube 特别难，因为与其他任务相比，它在放置过程中需要更精确的抓取和对齐，并且在异构多实施数据上进行训练时，这一挑战变得更加严峻。



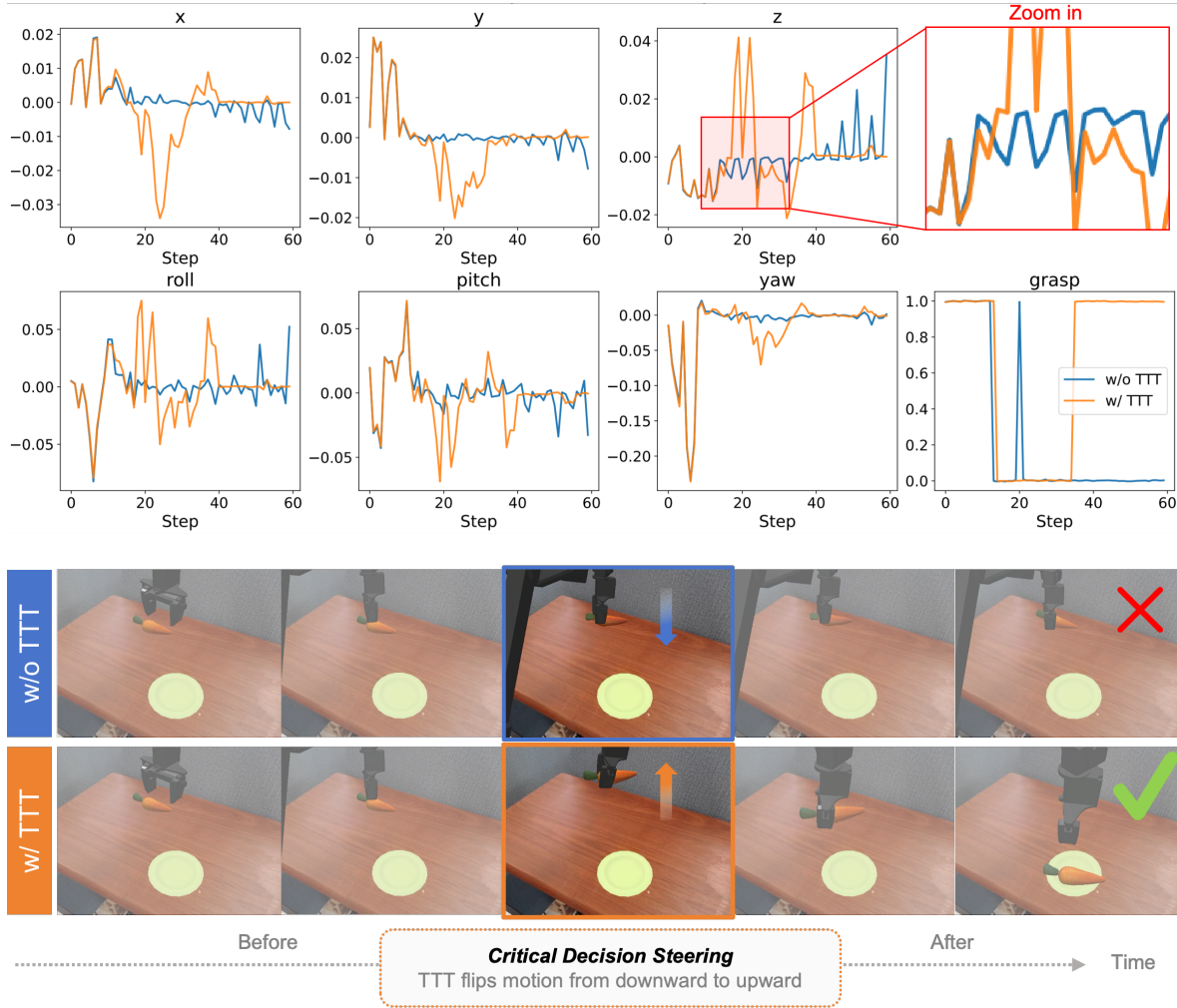


图 5 测试时训练效果的定性可视化。顶行比较了有和没有 TTT 的动作轨迹，底行显示了一个代表性的操作序列，其中 TTT 将关键决策从向下运动转向向上纠正运动，从而导致任务成功。

### 4.3 消融与分析

在线 TTT 与离线 TTT 在这里，在线 TTT 是指随着交互样本到达而不断更新潜在提示，使用批量大小 1 和学习率  $1 \times 10^{-7}$ 。该设置在[32, 34]中概念性地介绍，其目标是适应本地流数据分布。相比之下，离线 TTT 将交互样本存储在缓冲区中，然后执行批量优化。在离线 TTT 中，我们进一步区分每个任务的优化和所有任务的联合优化。如图6所示，我们发现完全在线提示更新比较敏感，导致性能大幅下降。因此，我们使用离线 TTT 作为主要设置，并将在线 TTT 视为稳定性分析而不是主要部署协议。同时，每个任务和联合离线优化之间的差异可以忽略不计。这些结果表明，批量大小（而不是任务分组）是关键因素，并且对于在测试时训练期间维持有意义的潜在表示可能很重要。稳定真正的动态即时更新仍然是未来的一个重要方向。

学习率敏感性我们还研究离线联合测试时训练的学习率敏感性，因为这个基本超参数对性能有很大影响。中等学习率  $10^{-5}$  表现最佳，优于较大更新规模  $10^{-4}$  和较小更新规模  $10^{-6}$ 。这一结果表明，仅即时适应并不是任意脆弱的，但它仍然受益于使用更新规模

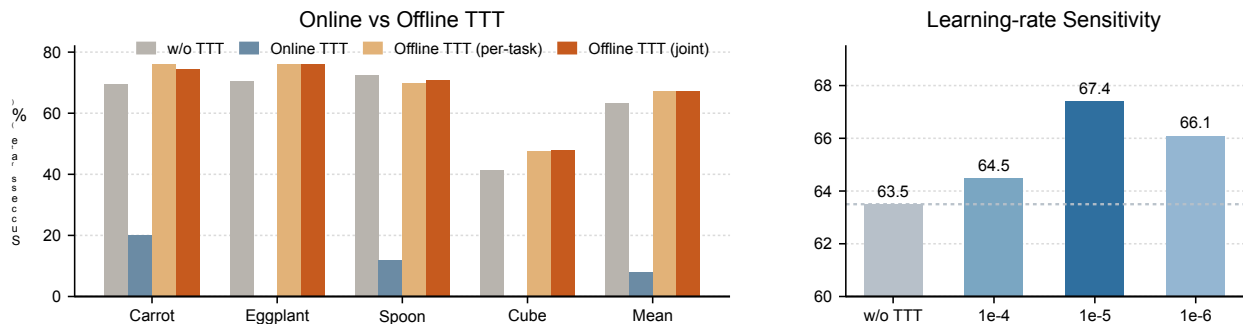


图 6 单实施例设置中测试时训练配置的分析。左：在线和离线 TTT 策略的任务比较，包括离线每个任务和离线联合变体。右图：离线联合 TTT 的学习率敏感性，其中  $10^{-5}$  的中等学习率可提供最佳平均成功率。

足够大以纠正特定于部署的不匹配，而不会破坏学习提示的稳定性。

收益从何而来。为了获得有效的比较，我们在种子完全固定的情况下重新运行实验<sup>1</sup>。这使我们能够仔细检查测试时训练改善行为的案例。我们发现大多数这些改进都遵循一个共同的模式，我们称之为“关键决策指导”。图 5 展示了一个代表性示例。在夹具抓住胡萝卜之前，有和没有 TTT 的动作输出在所有维度上几乎相同。抓握后立即出现分歧：没有 TTT 的模型不断向下移动，就好像它没有意识到胡萝卜已经固定并且手臂被桌子挡住了，而有 TTT 的模型反而适当地抬起了手臂。从这个决策点开始，两次部署将分开并且不再重叠。我们观察到跨任务的类似行为。许多操作任务由多个阶段组成，阶段之间的转换通常对应于关键决策点。一个常见的例子是判断抓取是否成功。在立方体任务中，在放置过程中出现了一个不同的关键点：在抓取立方体之后，策略必须在释放之前决定对象是否充分对齐。附录图 7 显示了另一个示例，其中测试时训练通过减少超调和产生更好的块对齐来改进此放置决策。

## 5 结论

我们研究了是否可以通过更新学习到的潜在提示来实现视觉-语言-动作模型的部署时间改进。为此，我们引入了 TTT-VLA，这是一个基于潜在提示优化的框架，它在训练期间学习潜在提示，并在部署时对冻结策略执行仅提示测试时训练。SimplerEnv 中的实验表明，该设计提高了单实施例和多实施例设置中的性能，包括在 WidowX 和 Google Robot 任务上的测试时训练中获得一致的收益。

实证分析进一步表明，测试时训练的好处通常来自于在少数关键决策点纠正行为，而不是全局改变整个行动轨迹。当前的限制是，在当前的代理任务下，这些改进主要采取对此类关键点附近的错误行为进行局部纠正的形式，而不是在较长时期内采取更复杂的纠正行为。这可能会限制仅提示测试时训练所能实现的改进上限。下一步重要的一步是开发更有效的提示优化程序，并研究该接口如何与更强大的 VLA 主干进行扩展。其他限制和悬而未决的问题在附录 .1 中讨论。

<sup>1</sup>This is not feasible for standard evaluation because flow matching needs random sampling

## 参考

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter 等。用于通用机器人控制的视觉-语言-动作流模型。arXiv 预印本 arXiv:2410.24164, 2024。
- [2] ~~Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu 等。~~ Rt-1: 用于大规模现实世界控制的机器人变压器。arXiv 预印本 arXiv:2212.06817, 2022。
- [3] ~~Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Chormanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn 等。~~ Rt-2: 视觉-语言-动作模型将网络知识转移到机器人控制。arXiv 预印本 arXiv:2307.15818, 2023。
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, 尼基尔·乔希, 瑞安·朱利安, 德米特里·卡拉什尼科夫, 邱宇恒, 伊莎贝尔·莱尔, 李光辉, 谢尔盖·莱文, 姚璐, 乌察夫·马拉, 迪克沙·曼均纳斯, 伊戈尔·莫达奇, 奥菲尔·纳胡姆, 卡罗莱纳·帕拉达, 乔迪林·佩拉尔塔, 艾米丽·佩雷斯, 卡尔·佩奇, 乔内尔·奎安宝, 卡尼什卡·拉奥, 迈克尔·柳, 格雷西亚·萨拉查, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu 和 Brianna Zitkovich。RT-1: 用于大规模现实世界控制的机器人变压器。在机器人学: 科学与系统 XIX。机器人技术: 科学与系统基金会, 2023 年 7 月。ISBN 978-0-9923747-9-2。doi: 10.15607/RSS.2023.XIX.025。
- 
- [5] 卜清文, 杨艳婷, 蔡继松, 高申元, 任光辉, 姚茂清, 罗平, 李红阳。Univla: 学习通过以任务为中心的潜在行动在任何地方采取行动, 2025 年。
- [6] Yossi Gandelsman, 孙宇, 陈新蕾, Alexei A. Efros。使用屏蔽自动编码器进行测试时训练。神经信息处理系统的进展, 2022 年。
- 
- [7] Nicklas Hansen, Rishabh Jangir, Yu Sun, Guillem Alenyà, Pieter Abbeel, Alexei A. Efros, Lerrel Pinto 和王小龙。部署期间自我监督的政策调整。arXiv 预印本 arXiv:2007.04309, 2020。
- 
- [8] 黄志平, 吴月华, 陈敏宏, 王玉强, 杨福恩。Thinkact: 通过强化视觉潜在规划进行视觉-语言-动作推理。arXiv 预印本 arXiv:2507.16815, 2025。
- 
- [9] 身体智能, 艾博, 阿里·阿明, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, 曹世豪, Thomas Charbonnier, 等。 $\pi_{0.7}$ : 具有紧急功能的可操纵多面手机人基础模型。arXiv 预印本 arXiv:2604.15483, 2026。
- 
- [10] 季冠华, Harsha Polavaram, Lawrence Yunliang Chen, Sandeep Bajamahal, Zehan Ma, Simeon Adebola, Chenfeng Xu, Ken Goldberg。Oxe-aug: 用于扩展跨实体策略学习的大规模机器人增强 oxe, 2025 年。
- [11] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Shaw, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang 和 Chelsea Finn。OpenVLA: 开源视觉-语言-动作模型, 2024 年 6 月。
- [12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Shaw, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang 和 Chelsea Finn。Openvla: 开源视觉-语言-动作模型。机器人学习会议, 第 270 卷, 第 2679-2713 页。PMLR, 2024 年。
- 
- [13] Moo Jin Kim, Yihuai Gau, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn 和 Jinwei Gu。Cosmos 政策: 微调视觉运动控制和规划的视频模型, 2026 年。
- [14] Jason Lee, 段家飞, 方浩泉, 邓玉泉, 刘硕, 李博阳, 方博涵, 张杰宇, 王一如, 李桑浩, Winson Han, Wilbert Pumaracay, Angelica Wu, Rose Hendrix, Karen Farley, Eli VanderBilt, Ali Farhadi, Dieter Fox 和 Ranjay Krishna。Molmoact: 可以在太空中推理的动作推理模型, 2025 年。
- [15] Pannag R. Sanketi Li, Hao Wang, Ajay Mandlekar, Sichen M. Yang, Haoran Geng, Yizhou Duan, Ruslan Li, Vincent Vanhoucke, Chelsea Finn, Julian Ibarz, Fei Xia 和 Tianhe Yu。在模拟中评估现实世界的机器人操纵策略。第八届机器人学习会议论文集, 2025 年。

[16] 李启秀, 梁耀波, 王泽宇, 罗琳, 陈希, 廖莫正, 魏方云, 邓宇, 徐思成, 张一中, 王晓凡, 刘蓓, 付建龙, 包建民, 陈东, 石元春, 杨蛟龙, 郭百宁。Cogact: 一种基础视觉-语言-动作模型, 用于协同机器人操作中的认知和动作。arXiv 预印本 arXiv:2411.19650, 2024。[17] 李世杰、李伟贤、周天乐、Abul Kalam Azad、刘建凯、龚业云、潘良明、李超、王张扬、安博和董立。Emma-x: 一种具体化的多模式行动模型, 具有扎根的思维链和前瞻空间推理。arXiv 预印本 arXiv:2412.11974, 2024。[18] Xinghang Li, Peiyan Li, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, tao Kong, Hanbo Zhang, and Huaping Liu. 走向通用机器人政策: 构建视觉-语言-动作模型的重要因素。arXiv 预印本 arXiv:2412.14058, 2024。[19] NVIDIA GEAR 团队、Johan Bjorck、~~Fernando Castañeda~~、Nikita Cherniadev、Xingye Da、Runyu Ding、Linxi Fan、Yu Fang、Dieter Fox、Fengyuan Hu、Spencer Huang、Joel Jang、Zhenyu Jiang、Jan Kautz、Kaushil Kundalia、Lawrence Lao、李志奇、林宗宇、林凯文、刘桂林, 等。Gr00t n1: 通用类人机器人的开放基础模型, 2025 年。[20] Octo 模型团队、Dibya Ghosh、Homer Walke、Karl Pertsch、Kevin Black、Oier Mees、Sudeep Dasari、Joey Hejna、Charles Xu、Jianlan Luo、Tobias Kreiman、You Liang Tan、Dorsa Sadigh、Chelsea Finn 和 Sergey Levine。Octo: 开源多面手机器人政策。https://octo-models.github.io, 2023。

[21] Abhishek Padalkar、Acorn Pooley、Ajinkya Jain、Alex Bewley、Alex Herzog、Alex Irpan、Alexander Khazatsky、Anant Rai、Anikait Singh、Anthony Brohan 等人。开放 x-embodiment: 机器人学习数据集和 rt-x 模型。arXiv 预印本 arXiv:2310.08864, 2023。

[22] Kaustubh Pathak、Lawrence Yunliang Chen、高一鸣、Somil Kent、Long Ma、Anca Dragan、Dorsa Sadigh 和 Chelsea Finn。Gr-rl: 灵巧而精确的长视野机器人操作, 2025 年。

[23] Karl Pertsch、Quan Vuong、Kevin Black、You Liang Tan、Adnan Esmail、Isabel Leal、Homer Walke、Chelsea Finn 和 Sergey Levine。快速: 视觉-语言-动作模型的高效动作标记化。arXiv 预印本 arXiv:2501.09747, 2025。

[24] 身体智力, 凯文·布莱克, 诺亚·布朗, 詹姆斯·达皮尼安, 卡兰·达巴利亚, 丹尼·德里斯, 阿德南·埃斯梅尔, 迈克尔·埃奎, 切尔西·芬恩, 尼科洛·福赛, 拉奇·格鲁姆, 卡罗尔·豪斯曼, 布莱恩·伊希特, 亚历克斯·伊尔潘, 朱利安·琼斯, 尼基尔·乔希, 德米特里·卡拉什尼科夫, 宇衡匡, 伊莎贝尔·莱尔, 谢尔盖·莱文, 姚路, 科里·林奇, 卡尔·Pertsch、Kanishka Rao、Krista Reymann、Michael Ryoo、Dorsa Sadigh、Grecia Salazar、Pannag Sanketi、Pierre Sermanet、Anikait Singh、Jost Tobias Springenberg、James Tanner、Quan Vuong、Homer Walke、Anna Walling、王浩欢、Lili Yu 和 Uryzhilinsky。π<sub>0.5</sub>: 具有开放世界泛化的视觉-语言-动作模型, 2025 年。

[25] 身体智力, Kevin Black、Noah Brown、Danny Driess、Adnan Esmail、Michael Equi、Chelsea Finn、Niccolo Fusai、Lachy Groom、Karol Hausman、Brian Ichter 等。π<sub>0.6</sub><sup>\*</sup>: 从经验中学习的 vla, 2025 年。

[26] 身体智力, Kevin Black、Noah Brown、Danny Driess、Adnan Esmail、Michael Equi、Chelsea Finn、Niccolo Fusai、Lachy Groom、Karol Hausman、Brian Ichter 等。π<sub>0.7</sub>: 具有紧急行为的可操纵多面手机器人基础模型, 2026 年。

[27] 屈德林, 宋浩明, 陈启智, 姚元琪, 叶欣怡, 丁彦, 王志刚, 顾佳元, 赵斌, 王东, 李雪龙。Spatialvla: 探索视觉语言动作模型的空间表示。arXiv 预印本 arXiv:2501.15830, 2025。[28] Lucy Xiaoyang Shi、Zheyuan Hu、Tony Z.Zhao、Archit Sharma、Karl Pertsch、Jianlan Luo、Sergey Levine 和 Chelsea Finn。对你的机器人大喊大叫: 通过语言纠正进行即时改进。载于《机器人学: 科学与系统》, 2024 年。[29] Yu Sun、Xiaolong Wang、Zhuang Liu、John Miller、Alexei A. Efros 和 Moritz Hardt。具有自我监督的测试时训练, 以实现分布变化下的泛化。第 37 届国际机器学习会议记录, 第 9229-9248 页, 2020 年。[30] Homer Walke、Kevin Black、Abraham Lee、Moo Jin Kim、Max Du、Chongyi Cheng、Tony Zhu、Philippe Hansen-Estruch、Quan Vuong、Andre He、Vivek Myers、Kuan Fang、Chelsea Finn 和 Sergey Levine。Bridgedata v2: 用于大规模机器人学习的数据集。机器人学习会议 (CoRL), 2023。[31] Chi Wan、Kangrui Wang、Yuan Si、Pingyue Zhang 和 Manling Li。Worldagen: 通过测试时世界模型训练进行统一的状态动作预测。AAAI 人工智能会议论文集, 2026 年。

[32] 王德全, Evan Shelhamer, 刘绍腾, Bruno Olshausen, Trevor Darrell. 帐篷: 通过熵最小化完全测试时间适应。学习表征国际会议, 2021. [33] Lirui Wang, Xinlei Chen, Jialiang Zhu, and Kaiming He. 使用异构预训练变压器扩展本体感觉视觉学习。神经信息处理系统进展, 2024 年. [34] 王钦、奥尔加·芬克、吕克·范·古尔和戴登新。持续测试时域适应。IEEE/CVF 计算机视觉和模式识别会议论文集, 第 7201–7211 页, 2022 年. [35] Yuqing Wen, Kefan Gu, Haoxuan Liu, Yu Cheng Zhu, Tiancai Wang, Haoqiang Fan 和 Xiaoyan Sun. Rosa: 利用机器人状态实现视觉语言和动作对齐, 2025. [36] 杨建伟、鲁本·谭、吴千惠、郑瑞杰、彭宝林、梁永元、谷宇、蔡木、叶成贤、乔尔·张、邓玉泉、拉斯·利登和高剑峰。Magma: 多模式人工智能代理的基础模型。arXiv 预印本 arXiv:2502.13130, 2025. [37] 杨帅, 李浩, 王斌, 陈逸伦, 田阳, 王泰, 王汉清, 赵峰, 廖依依, 庞江苗。Instructvla: 从理解到操作的视觉-语言-动作指令调整。arXiv 预印本 arXiv:2507.17520, 2025. [38] Seonghyeon Ye, Joel Jang, Byungguk Jeon, SeJune Joo, Jianwei Yang, Baolin Peng, Ajay Mandl ekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gau, Luke Zettlemoyer, Dieter Fox 和 Minjoon SEO. 视频潜在动作预训练, 2025. [39] Seonghyeon Ye, Yunhao Ge, Kaiyuan Cheng, Shenyuan Gau, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Shang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Cheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan 和 Joel Jang. 世界行动模型是零射击政策, 2026. [40] 赵清清, 路耀, 金武进, 付子鹏, 张卓阳, 吴业成, 李兆硕, 马千里, 韩松, 切尔西·芬恩, 等。Cot-vla: 视觉-语言-动作模型的视觉思维链推理。arXiv 预印本 arXiv:2503.22020, 2025. [41] 郑瑞杰、梁永源、黄帅一、高剑峰、Hal Daumé III、Andrey Kolobov、Furong Huang 和 Jenwei Yang. Tracevla: 视觉跟踪提示增强了通用机器人策略的时空意识。arXiv 预印本 arXiv:2412.10345, 2024.

## 附录

表 5 VLA 部署时间改进策略的扩展比较。该附录版本扩展了主表比较，并添加了额外的实际考虑因素，包括每个策略系列的典型优势和局限性。TTT-VLA 属于自我监督的 TTT 机制：它仅优化潜在提示，同时保持策略主干冻结。

|                                     | Human-Assisted Steering                                                                          | RL Post-Training                                                        | Test-time Training                                                                                               |
|-------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Intervention interface</b>       | Prompts, e.g., task-factorized instruction, metadata, trace                                      | Typically policy itself                                                 | Typically parts of the parameters. In this paper, we only use latent prompts.                                    |
| <b>Improvement signal</b>           | Human guidance / external signal                                                                 | Reward                                                                  | Self-supervised loss                                                                                             |
| <b>Policy backbone status</b>       | Frozen                                                                                           | Generally updated                                                       | Frozen                                                                                                           |
| <b>Typical strength</b>             | Simple and controllable at deployment time                                                       | Can improve beyond the pre-trained policy through continued interaction | Lightweight self-supervised improvement without updating the backbone                                            |
| <b>Typical limitation</b>           | Depends on human or externally specified guidance and is often bounded by the pre-trained policy | Usually costly, reward-dependent, and optimization-heavy                | Depends on the quality of the proxy objective, and gains may remain local or bounded by the adaptation interface |
| <b>Expected improvement pattern</b> | Mostly steers behavior within the capability envelope of the pretrained policy                   | Can reshape policy behavior more substantially through optimization     | Often corrects deployment-specific mistakes while preserving most pretrained behavior                            |

### .1 系统性限制

没有实际部署评估。我们尚未在完全真实的部署中评估 TTT-VLA。这种限制应该在问题设置的背景下理解：与标准模仿学习评估不同，现实世界的测试时训练需要在适应之前收集目标环境中的新交互数据。从这个意义上说，挑战更接近现实世界的强化学习部署，其中交互成本、系统稳定性和协议设计成为问题本身的一部分。由于VLA的测试时训练仍处于早期阶段，许多实际问题仍然悬而未决，包括如何构建数据收集、如何在部署过程中控制适应风险以及如何定义稳定的现实世界评估程序。因此，我们将严格的现实世界 TTT 评估视为超越当前基于模拟器的研究的一个重要但独立的挑战。

有限的代理任务比较。在当前的研究中，我们仅使用状态接地代理任务来实例化潜在提示优化。其他代理任务也可能很有前途，包括未来状态预测、未来图像预测、未来表示预测或逆动态预测。这些替代方案中的每一种都可能带来不同的优化挑战。例如，未来状态预测可能需要小心避免与主要行动预测目标混淆。未来图像预测可能是一项具有挑战性的任务，需要一个预先训练的世界模型作为预测头。未来的表示预测需要找到一个跨 DINO、SigLIP 等的有效编码器。我们首先专注于为 VLA 建立稳定且可观察的 TTT 设置，包括基本的超参数选择、自定义模拟器评估规则和个性化的部署时优化协议。这种稳定工作消耗了大量的实验时间，因此更广泛的代理任务比较仍然是未来的工作。

没有与之前的 TTT 方法进行直接比较。我们不提供与之前的 TTT 操纵策略方法的直接实验比较。具体来说，对于 VLA，最接近的并发工作是 WorldAgen [31]，



表 6 我们的实验中使用的默认测试时训练配置。除非另有说明，TTT-VLA 使用以下设置进行部署时提示优化。

| Item                          | Value                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------|
| TTT mode                      | Offline by default; online TTT is only used as an ablation                                      |
| Interaction buffer            | 800 / $\sim 800$ / $\sim 2000$ for WidowX/GR VM/ GR VA                                          |
| Trajectory source             | Rollouts collected from the pretrained policy                                                   |
| Success / failure composition | Includes both successful and failed rollouts                                                    |
| Optimization target           | State grounding loss only                                                                       |
| Updated parameters            | Latent prompt only                                                                              |
| Frozen parameters             | Vision-language backbone, action expert, and state expert                                       |
| TTT steps                     | 500 for WidowX; 1000 for Google Robot                                                           |
| Batch size                    | 128                                                                                             |
| Learning rate                 | $1 \times 10^{-5}$                                                                              |
| Wall-clock cost               | Approximately 15/30 minutes for WidowX/GR optimization                                          |
| Checkpoint selection          | Fixed optimization horizon for each benchmark; no best-step selection on evaluation performance |

表 7 在单实施例 WidowX 设置中 TTT 之前和之后的情节对齐结果比较。我们根据事件身份来调整基线和 TTT 后的部署，并比较二元任务结果。结果表明，TTT 并不是简单地修复固定的少数案例；而是修复少数案例。相反，它在配对部署中产生了净积极的转变，既有恢复的故障，也有偶尔的回归。

| Task     | Total | Base Succ. | Post Succ. | $\Delta$ | Succ. $\rightarrow$ Fail. | Fail. $\rightarrow$ Succ. | Always Succ. | Always Fail. |
|----------|-------|------------|------------|----------|---------------------------|---------------------------|--------------|--------------|
| Carrot   | 200   | 139        | 149        | +10      | 28                        | 38                        | 111          | 23           |
| Eggplant | 200   | 141        | 152        | +11      | 29                        | 40                        | 112          | 19           |
| Spoon    | 200   | 145        | 142        | -3       | 28                        | 25                        | 117          | 30           |
| Cube     | 200   | 83         | 96         | +13      | 29                        | 42                        | 54           | 75           |
| Overall  | 800   | 508        | 539        | +31      | 114                       | 145                       | 394          | 147          |

但其代码未公开，并且不报告 SimplerEnv 结果，使得受控比较变得困难。早期的工作，例如部署期间的自我监督策略适应 [7]，研究了部署时自我监督适应，但它早于现代 VLA 模型，因此在模型类别或评估设置中不具有直接可比性。

在线 TTT 的稳定性。我们观察到，当优化批量较小时，完全在线提示更新可能会不稳定，有时会导致提示崩溃。为此，我们使用离线 TTT 作为本文的主要部署协议。通过近端约束、信任域目标、即时规范正则化或微批量积累来稳定在线更新是未来工作的重要方向。

部署成本。TTT-VLA 需要收集目标环境中的交互轨迹并在评估之前进行及时优化。尽管只更新了少量参数，并且策略骨干保持冻结状态，但交互和优化成本并不为零。因此，我们明确报告轨迹的数量、优化步骤和挂钟开销，以便除了最终的性能增益之外，这种权衡也是可见的。



图 7 多维数据集任务中关键决策指导的其他示例。本附录图用另一个定性案例对图 5 进行了补充。在做出关键的安置决定之前，有和没有测试时训练的轨迹是相似的。临近发布时，测试时训练减少了向右超调，改善了块对齐，并导致成功的堆栈。

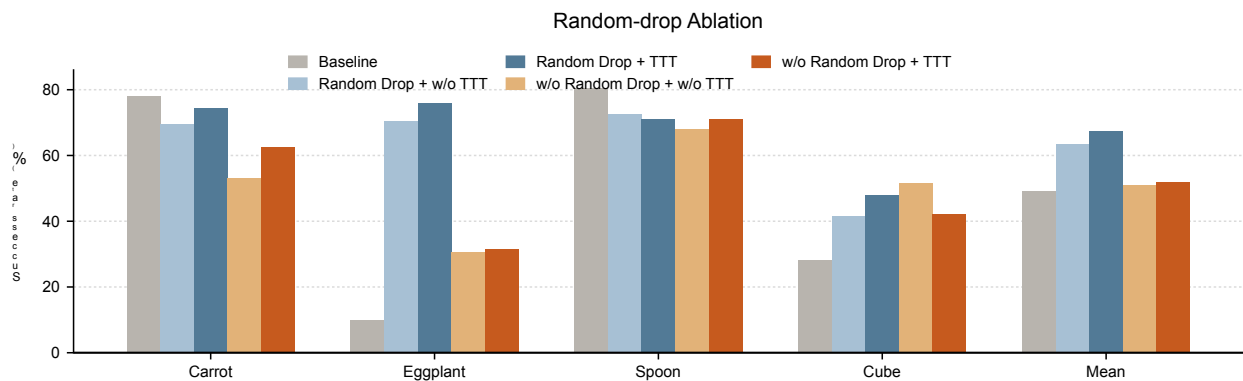


图 8 在单实施例设置中消除潜在提示优化的随机丢弃训练。我们比较了胡萝卜、茄子、勺子、立方体的原始基线、随机下降变体和没有随机下降的相应变体以及总体平均值。随机下降训练持续改进相应的变体，尤其是在测试时训练之后。